

Teoria współbieżności

Od struktur Kripkego do process mining

Plan 15 wykładów i 15 ćwiczeń po 90 minut

Punkty do omówienia, otwarte materiały, zadania i niezależna recenzja programu

Założony poziom: studenci informatyki po kursie automatów i podstaw logiki; kurs magisterski albo zaawansowany licencjacki.

Główna narracja: semantyka przeplotowa → częściowe porządki → sieci Petriego → weryfikacja → workflow i uczenie procesów → abstrakcja i granice rozstrzygalności.

Stan opracowania: materiały i linki zweryfikowane 31 lipca 2026 r.

Spis treści

1 Rekomendacja w skrócie

Plan jest merytorycznie mocny i może utworzyć wyjątkowo spójny kurs, pod warunkiem zachowania trzech ograniczeń:

1. LTL i CTL należy wykładać jako język specyfikacji oraz przykład model checkingu, a nie jako pełny kurs logik temporalnych.
2. Spośród logik na śladach Mazurkiewicza jedna powinna zostać omówiona dokładniej, a pozostałe jedynie jako mapa wyników.
3. Reachability dla ogólnych sieci Petriego powinno pozostać wynikiem orientacyjnym; rdzeń algorytmiczny mają stanowić coverability, WQO, inwarianty, równanie stanu i unfolding.

Kurs warto oprzeć na dwóch powracających przykładach:

- **protokół wzajemnego wykluczania** — dla LTS, logik temporalnych, bisymulacji, eksplozji przepływów, inwariantów i unfoldingów;
- **obsługa zamówienia lub wniosku kredytowego** — dla BPMN, workflow nets, logów zdarzeń, Declare, regionów, Inductive Minera i alignments.

1.1 Zakładane efekty uczenia

Po kursie student powinien umieć:

1. zbudować LTS lub strukturę Kripkego i zapisać podstawowe własności w LTL/CTL;
2. wykazać lub obalić symulację i bisymulację oraz skonstruować formułę Hennessy’ego–Milnera rozróżniającą stany;
3. przejść między słowem, klasą śladu, grafem zależności, pomsetem i częściowym porządkiem zdarzeń;
4. modelować współbieżność w sieci Petriego oraz odróżniać konflikt, przyczynowość i współbieżność;
5. stosować coverability, inwarianty, równanie stanu, pułapki, syfony i unfolding do analizy modelu;
6. analizować soundness workflow net oraz rozumieć relację BPMN–sieci Petriego;
7. wyjaśnić i ręcznie wykonać małe przykłady Declare mining, region-based mining i Inductive Miner;
8. interpretować fitness, precision i alignment;
9. rozumieć rolę abstrakcji, CEGAR oraz granice rozstrzygalności równoważności behawioralnych.

1.2 Plan semestru

Nr	Wykład	Główny rezultat
1	LTS i struktury Kripkego	model wykonania; safety i liveness
2	LTL, CTL i model checking	różnica czasu liniowego i rozgałęzionego
3	Symulacja, bisymulacja, HML	abstrakcja zachowująca obserwowalne zachowanie
4	Kompozycja i semantyka przepływowa	źródło eksplozji stanów i utraty niezależności

5	Posety i ślady Mazurkiewicza	reprezentacja współbieżności bez arbitralnych przepływów
6	Logiki na śladach	specyfikacja bezpośrednio na strukturze przyczynowej
7	Sieci Petriego	model zasobów, konfliktu i współbieżności
8	Occurrence nets i unfolding	częściowo uporządkowana semantyka wykonań
9	Techniki algebraiczne i CE-GAR	tanie nadaproksymacje i ich uszczegóławianie
10	Coverability i WQO	główny pełny blok algorytmiczny dla sieci
11	BPMN, workflow nets i soundness	zastosowanie formalnego modelu do procesów biznesowych
12	Logi zdarzeń i Declare	deklaratywne uczenie i kontrola procesów
13	Region-based process mining	synteza miejsc sieci z zachowania
14	Inductive Miner i conformance	strukturalne odkrywanie procesu i diagnoza odchyłań
15	Abstrakcja i nierozstrzygalność	spektrum równoważności i granice automatycznej analizy

2 Wykład 1: LTS, struktury Kripkego, wykonania

2.1 Cele

Student rozróżnia system przejść etykietowanych od struktury Kripkego, potrafi wyznaczyć ścieżki maksymalne oraz odróżnia własności stanowe, ścieżkowe, safety i liveness.

2.2 Plan 90 minut

0–10	Motywujący przykład: dwa wątki, wspólna zmienna i błąd zależny od kolejności.
10–30	LTS: stany, akcje, przejścia, ślady, wykonania skończone i nieskończone.
30–45	Struktury Kripkego i etykietowanie stanów; przejście między LTS a Kripke.
45–65	Własności stanów i ścieżek; invariance, reachability, deadlock freedom.
65–80	Safety i liveness; złe prefiksy i intuicja topologiczna bez formalizmu.
80–90	Mapa kursu: od jednej ścieżki do rozgałęzień i współbieżności.

2.3 Punkty do omówienia

- LTS $\mathcal{T} = (S, Act, \rightarrow, I)$ i notacja $s \xrightarrow{a} s'$.
- Ślad jako projekcja wykonania na etykiety; różnica między śladem a wykonaniem stanowym.
- Niedeterminizm jako wiele możliwych kontynuacji, nie jako losowość.
- Struktura Kripkego $K = (S, R, L, I)$ i totalizacja relacji dla semantyki nieskończonych ścieżek.
- Własności: osiągalność, nieosiągalność, brak zakleszczenia, sprawiedliwość.
- Safety: każde naruszenie posiada skończony świadek; liveness: żaden skończony prefiks nie przesądza porażki.
- Model jako abstrakcja: dobór stanu jest pierwszą decyzją weryfikacyjną.

2.4 Materiały

- **G. Winskel, Topics in Concurrency, §§1–2** — zwarty kurs od systemów przejść do modeli współbieżności; zawiera przykłady i ćwiczenia.
- **Cambridge, Exercises for Model Checking, §§1–2** — bardzo dobry bank zadań o modelowaniu systemów przejść.

- **C. Baier, J.-P. Katoen, Principles of Model Checking – materiały kursowe RWTH**
— strona kursu z wykładami i ćwiczeniami; poziom uzupełniający.

2.5 Ćwiczenia 90 minut

- 1.1 Modelowanie systemu przejść [E].** Zbudować LTS programu z dwoma licznikami i trzema instrukcjami warunkowymi. Jawnie określić, które informacje muszą należeć do stanu. Adaptacja: Cambridge, zad. 2.3–2.4.
- 1.2 Dziewięć przełączników [M].** Sformalizować stan jako wektor bitów, przejście jako operację XOR, a cel jako zbiór stanów. Zaproponować redukcję symetrii. Adaptacja: Cambridge, zad. 2.8.
- 1.3 Safety czy liveness? [M].** Dla ośmiu własności protokołu sklasyfikować je jako safety, liveness, obie lub żadna; dla safety podać zły prefiks.
- 1.4 Granica skończonej eksploracji [H].** Podać dwie nieskończone struktury o identycznych prefiksach do głębokości k , ale różniące się własnością liveness. Wyjaśnić, dlaczego bounded model checking wymaga dodatkowego argumentu kompletności.

Rezultat ćwiczeń: jedna strona formalnej specyfikacji LTS oraz zestaw świadków naruszenia safety. Nie używać jeszcze LTL; studenci mają najpierw zobaczyć, co logika będzie miała opisywać.

3 Wykład 2: LTL, CTL i podstawy model checkingu

3.1 Cele

Student zapisuje typowe wymagania w LTL i CTL, rozumie różnicę między kwantyfikacją po ścieżkach a operatorami temporalnymi oraz wykonuje algorytm etykietowania dla CTL na małym modelu.

3.2 Plan 90 minut

0–12	Wymagania słowne i ich niejednoznaczność.
12–35	Składnia i semantyka LTL: X, F, G, U ; nieskończone słowa.
35–53	CTL: formuły stanowe, kwantyfikatory A, E i pary operatorów.
53–68	Przykłady własności rozróżniających LTL i CTL.
68–82	Bottom-up model checking CTL jako obliczanie punktów stałych.
82–90	LTL model checking: tylko mapa redukcji do automatu Büchiego.

3.3 Punkty do omówienia

- Semantyka X, U oraz definicje $F\varphi$ i $G\varphi$.
- Wzorce: response, precedence, persistence, recurrence.
- Różnica między $AF p$, $EF p$, $AG EF p$ i $EG p$.
- Przykład rozdzielający FGp od $AFAGp$.
- Obliczanie $\text{Sat}(EX\varphi)$, $\text{Sat}(E[\varphi U \psi])$ i $\text{Sat}(EG\varphi)$.
- Punkty stałe najmniejsze i największe — intuicja, bez pełnego rachunku μ .
- Kontrprzykład jako ścieżka lub drzewo świadków.

3.4 Materiały

- [Stanford CS257, Lecture 5: Temporal Logic and Model Checking](#) — dobre slajdy do pierwszego kontaktu z LTL/CTL.
- [Cambridge, Exercises for Model Checking, §§5–7](#) — zadania o semantyce, rozróżnianiu formuł i model checkingu.
- [TU Eindhoven, Exercises Algorithms for Model Checking](#) — zadania z algorytmu etykietowania CTL i sprawiedliwości.
- [P. Gastin, Model Checking of CTL](#) — materiał uzupełniający; bardziej algorytmiczny.

3.5 Ćwiczenia 90 minut

- 2.1 Tłumaczenie wymagań [E].** Zapisać w LTL i, gdy to możliwe, w CTL: wzajemne wykluczanie, brak zagłodzenia, możliwość restartu, nieuchronne zakończenie. Wskazać założenia o sprawiedliwości.
- 2.2 Ręczna semantyka [M].** Dla pięciostanowej struktury wyznaczyć zbiory stanów spełniających EFp , AFp , $EG\neg p$, $AG(p \rightarrow AFq)$.
- 2.3 Algorytm etykietowania CTL [M].** Wykonać kolejne iteracje punktu stałego dla $E[pUq]$ i EGp . Adaptacja: TU Eindhoven, zad. 2.
- 2.4 Formuły rozdzielające [H].** Skonstruować model, w którym FGp i $AFAGp$ mają różne wartości; następnie wyjaśnić typ błędu wynikający z mylenia semantyki liniowej i rozgałęzionej. Adaptacja: Cambridge, zad. 7.2.

4 Wykład 3: Symulacja, bisymulacja i logika Hennessy’ego–Milnera

4.1 Cele

Student rozpoznaje relację symulacji i bisymulacji, stosuje grę bisymulacyjną, buduje iloraz oraz rozumie logiczną charakterystykę bisymulacji.

4.2 Plan 90 minut

0–15	Dlaczego równość języków jest za słaba dla systemów interaktywnych.
15–32	Symulacja jako preorder implementacja–specyfikacja.
32–52	Bisymulacja, największy punkt stały i gra Spoiler–Duplicator.
52–67	Algorytm dzielenia partycji dla skończonych LTS.
67–82	Hennessy–Milner logic i formuły rozróżniające.
82–90	Ilorazowanie i warunek skończonego rozgałęzienia.

4.3 Punkty do omówienia

- Trace equivalence, simulation equivalence i bisimilarity — pierwsza część spektrum liniowe–rozgałęzione.
- Symulacja jako relacja kierunkowa; refleksywność i przechodniość.
- Bisymulacja jako symetryczne dopasowanie kroków.
- Gra bisymulacyjna i konstrukcja krótkiego kontrprzykładu.
- HML: $\langle a \rangle \varphi$, $[a] \varphi$, negacja i koniunkcja.
- Twierdzenie Hennessy’ego–Milnera dla image-finite LTS.

- Minimalizacja przez bisymulację i zachowanie formuł modalnych.

4.4 Materiały

- **G. Winskel, Topics in Concurrency, rozdz. 5** — definicje, przykłady, ćwiczenia i połączenie z logiką modalną.
- **P. Sobociński, Bisimulation, Games, and Hennessy–Milner Logic** — bardzo użyteczny zestaw slajdów z maszyną do kawy, grami i formułami rozróżniającymi.
- **J. Pichon-Pharabod, Exercises for Model Checking, zad. 2.6–2.8** — symulacja jako preorder i abstrakcja przez symetrię.
- **P. Panangaden, Labelled Transition Systems** — uzupełnienie o koindukcję i logiczną charakterystykę.

4.5 Ćwiczenia 90 minut

- 3.1 Relacje na dwóch LTS [E].** Wyznaczyć największą symulację w obu kierunkach i sprawdzić, czy simulation equivalence implikuje bisymulację w danym przykładzie.
- 3.2 Gra bisymulacyjna [M].** Dla trzech par stanów podać strategię Duplicatora albo najkrótszą strategię wygrywającą Spoilera.
- 3.3 Formuła rozróżniająca [M].** Z wygrywającej strategii Spoilera skonstruować formułę HML prawdziwą w jednym stanie i fałszywą w drugim.
- 3.4 Iloraz [H].** Wykonać ręcznie refinement partycji, zbudować iloraz bisymulacyjny i sprawdzić, że zachowuje zestaw wskazanych formuł HML. Adaptacja: Winskel, ćwiczenia do rozdz. 5.

5 Wykład 4: Kompozycja równoległa i problem semantyki przepłotowej

5.1 Cele

Student buduje produkt synchroniczny/asynchroniczny LTS, rozumie reguły SOS oraz potrafi wskazać, które różnice w przestrzeni stanów są jedynie artefaktem przeplotu niezależnych akcji.

5.2 Plan 90 minut

0–15	Dwa komponenty i eksplozja liczby stanów.
15–35	Operatory: prefix, wybór, kompozycja równoległa, synchronizacja, ukrywanie.
35–52	Reguły SOS i generowanie globalnego LTS.
52–68	Diamant niezależności; ab i ba prowadzące do tego samego stanu.
68–82	State explosion i liczba liniaryzacji częściowego porządku.
82–90	Zapowiedź śladów Mazurkiewicza i unfoldingów.

5.3 Punkty do omówienia

- Produkt komponentów jako formalny model kompozycji.
- Synchronizacja handshake versus niezależny krok lokalny.
- Ukrywanie i akcja τ jako źródło weak equivalences.
- Reguły SOS jako definicja kompozycyjnej semantyki.
- Diamant komutacji: warunek dynamiczny i statyczna relacja niezależności.

- Dwie niezależne sekwencje długości m, n mają $\binom{m+n}{m}$ przeplotów.
- Problem: LTS pamięta kolejność liniową, ale nie mówi, że dwa kroki były niezależne.

5.4 Materiały

- **G. Winskel, Topics in Concurrency, §§2–4** — procesy, CCS/SOS, kompozycja i przykłady.
- **G. Winskel, M. Nielsen, Models for Concurrency** — klasyczny przegląd modeli przeplotowych i true concurrency.
- **J. Esparza, Unfolding Based Model Checking, slajdy 2–14** — bardzo czytelne porównanie produktu systemów i częściowo uporządkowanego rozwinięcia.

5.5 Ćwiczenia 90 minut

- 4.1 Produkt dwóch komponentów [E].** Z dwóch trzy-stanowych LTS zbudować produkt asynchroniczny i produkt z jedną synchronizacją.
- 4.2 Reguły SOS [M].** Wyprowadzić drzewa dowodowe dla czterech przejść procesu $(a.P + b.Q) \parallel_{\{b\}} R$.
- 4.3 Eksplozja przeplotów [M].** Policzyc liczbę ścieżek dla k niezależnych komponentów wykonujących po dwa kroki; porównać z liczbą zdarzeń i relacji w jednym pomsecie.
- 4.4 Diament i niezależność [H].** Sformułować wystarczające warunki, przy których $s \xrightarrow{a} s_1 \xrightarrow{b} s'$ można zamienić na $s \xrightarrow{b} s_2 \xrightarrow{a} s'$. Podać kontrprzykład, gdy akcje współdzielą zasób.

6 Wykład 5: Posety, pomsety i ślady Mazurkiewicza

6.1 Cele

Student przechodzi między słowem, klasą komutacyjną, grafem zależności i postacią Foaty oraz rozumie, które informacje o współbieżności zachowuje ślad.

6.2 Plan 90 minut

0–12	Przypomnienie częściowych porządków, pokryć i diagramów Hassego.
12–28	Pomsety: etykietowane posety modulo izomorfizm.
28–45	Alfabet zależności (Σ, I) i kongruencja generowana przez $ab = ba$.
45–62	Graf zależności jako kanoniczna reprezentacja śladu.
62–75	Postać normalna Foaty i maksymalne kroki równoległe.
75–90	Związek śladów z wykonaniami sieci Petriego; ograniczenie: brak rozgałęziającego konfliktu.

6.3 Punkty do omówienia

- Poset, liniowe rozszerzenie, antyłańcuch i szerokość.
- Pomset jako jedno wykonanie z przyczynowością, nie jako cały system z wyborem.
- Niezależność I : symetryczna i przeciwzwrotna; zależność $D = (\Sigma \times \Sigma) \setminus I$.
- Równoważność słów przez zamianę sąsiednich niezależnych liter.
- Graf zależności: wierzchołki jako wystąpienia zdarzeń, krawędzie jako minimalne wymuszenia kolejności.

- Postać Foaty jako sekwencja klik niezależności.
- Trace-closed language i domknięcie względem komutacji.
- Dlaczego jeden ślad nie reprezentuje konfliktu pomiędzy alternatywnymi zdarzeniami.

6.4 Materiały

- **V. Diekert, A. Muscholl, Trace Theory** — najlepszy zwięzły tekst referencyjny: monoid śladów, grafy zależności, automaty.
- **MIM UW, Ślady Mazurkiewicza** — slajdy odpowiednie bezpośrednio do wykładu.
- **A. Maarand, T. Uustalu, Certified Foata Normalization** — precyzyjne definicje i algorytm normalizacji; materiał opcjonalny.
- **M. Koutny, Generalising Mazurkiewicz Traces** — wizualne przykłady relacji między wykonaniami a porządkami przyczynowymi.

6.5 Ćwiczenia 90 minut

Niech $\Sigma = \{a, b, c, d\}$ i $I = \{(a, c), (c, a), (a, d), (d, a), (b, d), (d, b)\}$.

- 5.1 Klasa śladu [E].** Wypisać wszystkie słowa równoważne $abacd$ i uzasadnić kompletność listy.
- 5.2 Graf zależności [M].** Zbudować graf zależności dla $abacd$, wykonać redukcję przechodnią i wypisać wszystkie liniowe rozszerzenia.
- 5.3 Postać Foaty [M].** Obliczyć postać normalną Foaty dwiema metodami: z grafu zależności i przez algorytm zachłanny; porównać.
- 5.4 Ślad a konflikt [H].** Podać dwa systemy o tym samym zbiorze maksymalnych pomsetów długości co najwyżej 2, ale różnej strukturze konfliktu. Wyjaśnić potrzebę event structures lub occurrence nets.

7 Wykład 6: Logiki na śladach Mazurkiewicza

7.1 Cele

Student rozumie trzy sposoby interpretowania logiki temporalnej dla częściowych porządków: na liniaryzacjach, globalnie na konfiguracjach oraz lokalnie na zdarzeniach. Potrafi rozpoznać własność niezmienniczą względem niezależnych zamian.

7.2 Plan 90 minut

0–15	Dlaczego zwykle LTL może zależeć od arbitralnej liniaryzacji.
15–32	Trace-closed specifications: uniwersalna i egzystencjalna interpretacja na liniaryzacjach.
32–52	Logiki lokalne na zdarzeniach i relacje następstwa procesowego.
52–68	Egzystencjalne i uniwersalne until na przedziałach częściowego porządku.
68–80	FO/MSO nad grafami zależności; twierdzenia o pełności ekspresyjnej jako mapa wyników.
80–90	Decyzja dydaktyczna: jedna logika w szczegółach, reszta jako panorama.

7.3 Punkty do omówienia

- Semantyka $t \models_{\forall} \varphi$ wtedy i tylko wtedy, gdy każda liniaryzacja śladu spełnia φ .
- Semantyka egzystencjalna i różnica interpretacyjna.
- Niezmienniczość formuły względem zamian liter niezależnych.
- Lokalny successor dla procesu/komponentu i modalności zdarzeniowe.
- Przedział $[e, f]$ w częściowym porządku; egzystencjalne versus uniwersalne until.
- FO nad zdarzeniami z porządkiem i predykatami etykiet.
- Wynik Thiagarajana–Walukiewicza: temporalna logika ekspresyjnie kompletna względem odpowiedniego FO — bez dowodu technicznego.
- Granica kursu: nie wprowadzać pełnej teorii automatów asynchronicznych Zielonki.

7.4 Materiały

- [M. Mukund, A Quick Tour of Trace Logics](#) — najlepszy materiał slajdowy: warianty logik lokalnych, diagramy i wyniki ekspresyjności.
- [P. S. Thiagarajan, I. Walukiewicz, An Expressively Complete Linear Time Temporal Logic for Mazurkiewicz Traces](#) — główne źródło teoretyczne do jednego reprezentatywnego wyniku.
- [V. Diekert, P. Gastin, Pure Future Local Temporal Logics are Expressively Complete for Mazurkiewicz Traces](#) — wariant bardziej lokalny; materiał do lektury seminaryjnej.
- [V. Diekert, A. Muscholl, Trace Theory, część logiczna](#) — kontekst algebraiczny i automatyczny.

7.5 Ćwiczenia 90 minut

- 6.1 Niezmienniczość względem śladu [E].** Dla pięciu formuł LTL sprawdzić, czy ich prawdziwość może zmienić zamiana dwóch sąsiednich niezależnych zdarzeń. Dla odpowiedzi negatywnej podać najmniejszy kontrprzykład.
- 6.2 Dwie semantyki liniaryzacji [M].** Dla jednego pomsetu obliczyć wartość $\models_{\forall} F(a \wedge Xb)$ i $\models_{\exists} F(a \wedge Xb)$. Wyjaśnić, dlaczego X jest szczególnie problematyczny.
- 6.3 Lokalne until [M].** Na podanym diagramie Hassego oznaczyć zdarzenia spełniające egzystencjalne i uniwersalne $\varphi U \psi$. Adaptacja z przykładów Mukunda.
- 6.4 FO versus temporalność [H].** Zapisać w FO własność: „każde a ma późniejsze, zależne przyczynowo b , przed którym nie ma c ”; następnie zaproponować odpowiadającą jej formułę lokalnej logiki temporalnej.

Kontrola zakresu: jest to najbardziej ryzykowny wykład semestru. W wersji podstawowej należy dokładnie zdefiniować tylko semantykę na liniaryzacjach i jedną logikę lokalną. Twierdzenia o pełności ekspresyjnej wystarczą jako 10-minutowa panorama.

8 Wykład 7: Sieci Petriego — składnia i semantyka

8.1 Cele

Student modeluje zasoby, synchronizację i konflikt w sieci P/T; wyznacza sekwencje odpaleń, znakowania osiągalne i relacje współbieżności.

8.2 Plan 90 minut

0–12	Dlaczego posety nie są jeszcze generatywnym modelem systemu.
12–32	Miejsca, przejścia, łuki, znakowanie i reguła odpalenia.
32–48	Sekwencje odpaleń i graf osiągalności.
48–63	Konflikt strukturalny i dynamiczny; współbieżne uaktywnienie.
63–76	Semantyka krokowa i komutacja niezależnych przejść.
76–90	Własności: boundedness, safeness, deadlock, liveness, reversibility.

8.3 Punkty do omówienia

- Sieć ważona i jej wariant zwykły; pre- i postset.
- Znakowanie jako wektor w $\mathbb{N}^P$.
- Monotoniczność odpaleń względem porządku składowego.
- Różnica: dwa przejścia uaktywnione razem, konflikt o token, prawdziwa współbieżność.
- Krok jako multizbiór przejść; sekwencyzacja kroku.
- Graf osiągalności dla sieci ograniczonej.
- Przykłady: producent–konsument, semafor, mutual exclusion, fork–join.

8.4 Materiały

- [TUM, Petri Nets — strona kursu](#) — kompletny kurs z notatkami, slajdami, ośmioma zestawami ćwiczeń i rozwiązaniami.
- [TUM, Petri Nets Lecture Notes](#) — podstawowe źródło do definicji i dalszych algorytmów.
- [TUM, Basic Petri Net Slides](#) — gotowe wizualne przykłady konfliktu, przyczynowości i własności sieci.
- [S. Schwoon, Petri Nets](#) — zwarty zestaw slajdów obejmujący podstawy, algebrę i unfolding.

8.5 Ćwiczenia 90 minut

- 7.1 Odpalenia i graf osiągalności [E].** Dla małej sieci wypisać wszystkie znakowania, przejścia uaktywnione i graf osiągalności. Adaptacja: TUM, arkusz 1, zad. 1.1.
- 7.2 Modelowanie producenta i konsumenta [M].** Zbudować sieć z buforem pojemności 2, a następnie wariant z nieograniczonym buforem; wskazać deadlock i boundedness.
- 7.3 Wilk, koza i kapusta [M].** Zamodelować problem jako bezpieczną sieć, ustalić znakowanie docelowe i znaleźć sekwencję. Adaptacja: TUM, arkusz 2.
- 7.4 Konflikt czy współbieżność [H].** Dla każdej pary przejść sklasyfikować relację strukturalną i dynamiczną oraz podać znakowanie rozróżniające oba pojęcia, jeżeli istnieje.

9 Wykład 8: Occurrence nets, konfiguracje i unfolding

9.1 Cele

Student interpretuje unfolding jako reprezentację wykonań bez redundancji przeplotów, wyznacza konfiguracje i cięcia oraz rozumie ideę cutoff event i complete finite prefix.

9.2 Plan 90 minut

0–15	Od jednego śladu do rozgałęzionej historii: potrzeba reprezentacji konfliktu.
15–32	Occurrence nets: warunki, zdarzenia, acykliczność i nierozgałęziająca się przeszłość.
32–48	Przyczynowość, konflikt dziedziczny i współbieżność.
48–63	Konfiguracje, cięcia i odpowiadające znakowania.
63–76	Konstrukcja unfoldingu małej sieci.
76–90	Cutoff, adequate order i complete finite prefix — intuicja algorytmu.

9.3 Punkty do omówienia

- Branching process i homomorfizm etykietujący do sieci pierwotnej.
- Lokalna konfiguracja zdarzenia.
- To samo przejście sieci może mieć wiele zdarzeń w unfoldingu z różnymi historiami.
- Konfiguracje jako skończone, przyczynowo domknięte i bezkonfliktowe zbiory zdarzeń.
- Cięcie odpowiada znakowaniu; równoległe zdarzenia nie generują osobnych gałęzi dla każdej kolejności.
- Cutoff przez powtórzenie reprezentowanego znakowania i adequate order.
- Co complete finite prefix zachowuje dla sieci bezpiecznych/ograniczonych.

9.4 Materiały

- [J. Esparza, Unfolding Based Model Checking](#) — 122 slajdy; najlepsze źródło do budowy wykładu i ilustracji.
- [S. Schwoon, Petri Nets, slajdy 50–75](#) — krótszy wariant z konstrukcją prefiksu krok po kroku.
- [J. Esparza, K. Heljanko, Unfoldings — A Partial-Order Approach to Model Checking](#) — pełny bezpłatny draft monografii; źródło referencyjne.
- [V. Khomenko, Model Checking Based on Prefixes of Petri Net Unfoldings](#) — pogłębienie algorytmiczne i zastosowania.

9.5 Ćwiczenia 90 minut

- 8.1 **Relacje w occurrence net** [E]. Dla podanej sieci zdarzeń wyznaczyć $<$, $\#$ i co dla wskazanych par.
- 8.2 **Konfiguracje i cięcia** [M]. Spośród ośmiu zbiorów zdarzeń wybrać konfigurację; dla każdej wyznaczyć cięcie i znakowanie sieci bazowej. Adaptacja: Esparza, slajdy o configurations.
- 8.3 **Ręczny unfolding** [M]. Rozwinąć sieć fork-join z konfliktem do głębokości dwóch powrotów i zaznaczyć powtarzające się znakowania.
- 8.4 **Cutoff i kompletność** [H]. Wybrać cutoff events w dwóch różnych kolejnościach dodawania zdarzeń; porównać rozmiar prefiksów i uzasadnić, że oba reprezentują wszystkie znakowania. Adaptacja: Schwoon, slajdy 53–67.

10 Wykład 9: Równanie stanu, inwarianty, pułapki, syfony i CEGAR

10.1 Cele

Student stosuje macierz incydencji i inwarianty do dowodzenia bezpieczeństwa, rozumie niezupełność równania stanu oraz wykonuje prostą iterację counterexample-guided abstraction refinement.

10.2 Plan 90 minut

0–15	Macierz incydencji i wektor Parikha sekwencji odpaleń.
15–32	Równanie stanu $M = M_0 + C\sigma$ jako warunek konieczny.
32–50	P-inwarianty i dowody boundedness/mutual exclusion.
50–63	T-inwarianty i ich ograniczona interpretacja behawioralna.
63–75	Pułapki i syfony jako informacja utracona przez macierz.
75–90	CEGAR: rozwiązanie liniowe, sprawdzenie realizowalności, refinement.

10.3 Punkty do omówienia

- $C = Post - Pre$; równanie stanu i wektor liczności odpaleń.
- Rozwiązanie równania nie ustala kolejności i nie gwarantuje uaktywnienia przejść.
- P-inwariant $y^T C = 0$ i zachowanie $y^T M$.
- Dodatni P-inwariant jako wystarczający argument strukturalnego ograniczenia.
- T-inwariant $Cx = 0$: potencjalny cykl efektów, niekoniecznie wykonywalny.
- Pułapka oznaczona pozostaje oznaczona; opróżniony syfon pozostaje pusty.
- Abstrakcja liniowa jako nadaproksymacja osiągalności.
- Refinement przez ograniczenia kolejnościowe i constraints on transition counts.

10.4 Materiały

- [TUM Petri Nets, arkusze 5 i 6](#) — gotowe zadania z równania stanu, P/T-inwariantów, pułapek, syfonów i mutual exclusion.
- [S. Schwoon, Petri Nets, slajdy 34–49](#) — bardzo klarowny przykład pokazujący, dlaczego równanie stanu jest tylko nadaproksymacją.
- [H. Wimmel, K. Wolf, Applying CEGAR to the Petri Net State Equation](#) — bezpośrednie źródło do połączenia algebry sieci Petriego z CEGAR.
- [E. Clarke et al., Counterexample-Guided Abstraction Refinement](#) — klasyczne tło CEGAR; tylko wybrane slajdy/pojęcia.

10.5 Ćwiczenia 90 minut

9.1 Równanie stanu [E]. Zbudować macierz incydencji, rozwiązać trzy równania dla wskazanych znakowań i zaznaczyć, które przypadki są natychmiast wykluczone. Adaptacja: TUM, arkusz 5, zad. 5.4.

9.2 Inwarianty [M]. Wyznaczyć bazę P-inwariantów i użyć jej do dowodu 1-boundedness oraz mutual exclusion. Adaptacja: TUM, arkusz 6, zad. 6.1 i 6.3.

9.3 Pułapka usuwa fałszywy świadek [M]. Dla rozwiązania równania stanu, które sugeruje niebezpieczne znakowanie, znaleźć oznaczoną pułapkę wykluczającą jego osiągalność.

9.4 Jedna iteracja CEGAR [H]. Rozwiązać abstrakcyjny problem ILP, odtworzyć potencjalną kolejność, wykryć blokujący prefix i dodać refinement. Porównać z algorytmem Wimmela–Wolfa na poziomie idei.

11 Wykład 10: Coverability, WQO i problemy podstawowe

11.1 Cele

Student odróżnia reachability od coverability, konstruuje drzewo/graf Karp–Millera i bazę w algorytmie wstecznym oraz rozumie rolę twierdzenia Dicksona w terminacji.

11.2 Plan 90 minut

0–12	Mapa problemów: reachability, coverability, boundedness, termination, deadlock.
12–30	Porządek składowy na $\mathbb{N}^d$ i twierdzenie Dicksona.
30–52	Karp–Miller: akceleracja przez ω i coverability tree/graph.
52–70	Algorytm wsteczny na zbiorach upward-closed i minimalnych bazach.
70–82	Redukcje: boundedness, uaktywnienie przejścia, deadlock w wariantach.
82–90	Reachability: rozstrzygalność i Ackermann-completeness tylko jako orientacja.

11.3 Punkty do omówienia

- M pokrywa M_f , gdy $M \geq M_f$; różnica względem dokładnej osiągalności.
- WQO: brak nieskończonych antyłańcuchów i nieskończonych ściśle malejących ciągów.
- Akceleracja komponentów rosnących do ω .
- Odczytywanie boundedness i coverable markings z drzewa Karp–Millera.
- $Pre(\uparrow B)$ i minimalizacja bazy w algorytmie wstecznym.
- Monotoniczność sieci jako warunek poprawności WSTS.
- Reachability nie powinno być wykładane algorytmicznie; warto podać jedynie status i skalę złożoności.

11.4 Materiały

- [TUM, Petri Nets Lecture Notes, rozdziały o coverability](#) — pełne i dydaktycznie sprawdzone omówienie obu algorytmów.
- [TUM, Exercise Sheet 4](#) — zadania z grafu pokrywalności i algorytmu wstecznego wraz z rozwiązaniami.
- [P. Schnoebelen, Algorithmic Aspects of WQOs](#) — szerszy kontekst WSTS i WQO.
- [J. Leroux, S. Schmitz, Reachability in Vector Addition Systems is Ackermann-complete](#) — aktualny punkt odniesienia dla złożoności reachability; tylko jedna plansza.

11.5 Ćwiczenia 90 minut

- 10.1 Karp–Miller [E].** Zbudować graf pokrywalności, wskazać miejsca ograniczone i opisać zbiór pokrywalnych znakowań. Adaptacja: TUM, arkusz 4, zad. 4.1.
- 10.2 Wpływ kolejności eksploracji [M].** Dla tej samej sieci zbudować dwa różne drzewa Karp–Millera wynikające z różnych strategii. Adaptacja: TUM, zad. 4.2.
- 10.3 Algorytm wsteczny [M].** Obliczać kolejne minimalne bazy B_i dla dwóch celów; jeden ma być pokrywalny, drugi nie. Adaptacja: TUM, zad. 4.3.
- 10.4 Dłaczego algorytm się kończy [H].** Sformułować dowód terminacji algorytmu wstecznego z użyciem WQO; wskazać dokładnie, gdzie potrzebne są monotoniczność i skończona baza predecessorów.

12 Wykład 11: BPMN, workflow nets i soundness

12.1 Cele

Student rozumie semantyczny rdzeń BPMN, potrafi przełożyć fragment procesu na workflow net oraz sprawdzić podstawowe warianty soundness i typowe błędy modelowania.

12.2 Plan 90 minut

0–12	Po co formalizować proces biznesowy; notacja a semantyka.
12–32	BPMN: events, tasks, sequence flow, XOR/AND gateways, pools i messages.
32–48	Ograniczony, formalizowalny fragment BPMN.
48–63	Workflow net: jedno wejście, jedno wyjście, każdy element na ścieżce $i \rightarrow o$.
63–78	Soundness: option to complete, proper completion, no dead transitions.
78–90	Short-circuit net, boundedness/liveness i przykłady weryfikacji.

12.3 Punkty do omówienia

- Rozdzielenie notacji BPMN od wykonawczej semantyki silnika workflow.
- XOR split/join, AND split/join i pułapka niezgodnego łączenia bramek.
- Message flow nie jest sequence flow.
- Workflow net i znakowania początkowe/końcowe.
- Klasyczne soundness oraz krótka wzmianka o relaxed/generalized soundness.
- Redukcja soundness do liveness i boundedness po short-circuit.
- Błędy: dead transition, deadlock, livelock, pozostawione tokeny.
- Narzędzia: WoPeD do wizualizacji i ProM/LoLA do analizy.

12.4 Materiały

- [OMG, BPMN 2.0.2 — oficjalna specyfikacja](#) — źródło normatywne; wybierać tylko niewielki fragment notacji.
- [BPMN.org, Documents and Examples](#) — oficjalne przykłady BPMN 2.0 i poster notacji.
- [A. Marrella, BPMN Lecture Slides](#) — gotowe slajdy dydaktyczne o konstrukcjach BPMN.

- **W. van der Aalst, The Application of Petri Nets to Workflow Management** — klasyczne wprowadzenie do workflow nets i soundness.
- **W. van der Aalst et al., Soundness of Workflow Nets** — źródło pogłębione do wariantów soundness.

12.5 Ćwiczenia 90 minut

- 11.1 Model BPMN [E].** Narysować proces obsługi zamówienia z równoległą kontrolą płatności i dostępności, anulowaniem oraz ponowieniem płatności.
- 11.2 BPMN → workflow net [M].** Przełożyć rdzeń modelu na sieć Petriego i wyjaśnić rolę każdego miejsca kontrolnego.
- 11.3 Diagnoza soundness [M].** Dla trzech workflow nets znaleźć odpowiednio: dead transition, deadlock i improper completion; podać minimalny ślad ujawniający błąd.
- 11.4 Naprawa modelu [H].** Minimalnie zmodyfikować wadliwy model tak, by stał się sound, i uzasadnić poprawność przez short-circuit oraz inwarianty.

13 Wykład 12: Logi zdarzeń, jakość modeli i Declare

13.1 Cele

Student rozumie strukturę logu zdarzeń, cztery główne wymiary jakości modelu oraz potrafi zapisać i sprawdzić podstawowe constraints Declare na skończonych śladach.

13.2 Plan 90 minut

0–15	Case, event, activity, timestamp, lifecycle; ślad i multizbiór śladów.
15–30	Trzy rodziny zadań: discovery, conformance, enhancement.
30–43	Fitness, precision, generalization, simplicity.
43–60	Model imperative versus declarative; założenie open world.
60–78	Declare: existence, response, precedence, succession, not-coexistence.
78–90	Discovery constraints: support, confidence i usuwanie redundancji.

13.3 Punkty do omówienia

- Log jako multizbiór skończonych śladów, a nie zbiór wszystkich możliwych zachowań.
- Problem niekompletności i szumu; brak śladu nie oznacza niemożliwości.
- Model flower: maksymalny fitness, bardzo niska precision.
- Model dokładnie odtwarzający log: ryzyko przeuczenia i słabej generalizacji.
- Declare jako zbiór constraints o semantyce LTL na skończonych słowach.
- Activations, fulfillments, violations i vacuous satisfaction.
- Mining jako selekcja instancji templates przekraczających progi.

13.4 Materiały

- **W. van der Aalst, Process Mining Courses and Materials** — centralny katalog wykładów, ćwiczeń, datasetów i nagrań.
- **W. van der Aalst, J. Carmona (red.), Process Mining Handbook** — otwarty podręcznik; rozdziały przeglądowe, discovery, conformance i declarative mining.
- **C. Di Ciccio, M. Montali, Declarative Process Specifications** — najpełniejsze źródło do semantyki, reasoning, discovery i monitoring Declare.
- **A. Marrella, Declarative Process Modeling and Mining** — gotowe slajdy z templates, formułami LTLf i przykładami śladów.
- **F. Maggi et al., Declare Component of ProM** — krótki opis narzędzia i pracy z rzeczywistym logiem.

13.5 Ćwiczenia 90 minut

- 12.1 Od tabeli do logu [E].** Z tabeli 20 zdarzeń utworzyć cases, variants i directly-follows counts; wskazać dwa problemy jakości danych.
- 12.2 Cztery wymiary jakości [M].** Porównać cztery modele dla tego samego logu: flower, enumerator śladów, model blokowy i model z jedną pętlą. Uporządkować je jakościowo według fitness, precision, generalization i simplicity.
- 12.3 Semantyka Declare [M].** Dla sześciu śladów ocenić ograniczenia $R(A, B)$, $P(A, B)$, $AltR(A, B)$ i $NC(A, B)$ (odpowiednio Response, Precedence, Alternate Response i Not Coexistence); jawnie omówić vacuity. Adaptacja: slajdy Marrelli.
- 12.4 Mały Declare Miner [H].** Dla logu policzyć support i confidence kandydatów z trzech templates, usunąć constraints logicznie implikowane i zinterpretować końcowy model.

14 Wykład 13: Region-based process mining

14.1 Cele

Student rozumie region jako kandydackie miejsce sieci, formułuje warunki separacji stanów i zdarzeń oraz ręcznie syntetyzuje małą sieć z transition systemu zbudowanego z logu.

14.2 Plan 90 minut

0–15	Dwa etapy: log $\rightarrow$ transition system $\rightarrow$ sieć Petriego.
15–32	Jak budować stany z prefixów, set/bag abstractions i horyzontu.
32–52	Region jako funkcja stanu i sygnatura zdarzeń; interpretacja miejsca.
52–67	State separation problem i event/state separation problem.
67–80	Synteza sieci z rodziny regionów; gwarancje i nadaproksymacja.
80–90	Wpływ niekompletnego logu i brakujących przepływów.

14.3 Punkty do omówienia

- Prefix transition system jako najprostsza reprezentacja całego logu.
- Abstrakcja stanu steruje generalizacją jeszcze przed syntezą sieci.
- Region $r : S \rightarrow \mathbb{N}$ oraz stały efekt etykiety na wszystkich jej krawędziach.
- Konsumpcja/produkcja tokenów jako sygnatura regionu dla zdarzenia.

- SSP: rozdzielenie dwóch różnych stanów; ESSP: zablokowanie etykiety nieobecnej w stanie.
- Zestaw regionów daje miejsca; krawędzie TS mają odpowiadać odpaleniom przejść.
- Dokładna synteza versus process discovery z niekompletnego logu.
- Związek z automatami skończonymi: „zbuduj automat, a następnie złoż/fold do sieci” jest trafną intuicją dydaktyczną, ale regiony nie są prostym scalaniem stanów.

14.4 Materiały

- **W. van der Aalst et al., A Two-Step Approach using Transition Systems and Regions** — najlepsze źródło bezpośrednio zgodne z narracją $\log \rightarrow TS \rightarrow PN$.
- **R. Bergenthum et al., Process Mining Based on Regions of Languages** — regiony języków i zastosowanie do process mining.
- **J. Carmona et al., A Region-Based Algorithm for Discovering Petri Nets** — algorytm generujący ograniczoną sieć nadaproksymującą log.
- **W. van der Aalst, Process Mining Lecture: Region-Based Mining** — nagranie wykładu z intuicją i przykładami.

14.5 Ćwiczenia 90 minut

Dany jest $\log L = [\langle a, b, d \rangle^2, \langle a, c, d \rangle^2]$.

- 13.1 Prefix transition system [E].** Zbudować deterministyczny prefix automaton/TS, a następnie wariant z abstrakcją „zbiór dotychczas wykonanych aktywności”.
- 13.2 Kandydackie regiony [M].** Znaleźć regiony odpowiadające: przed a , pomiędzy a a wyborem b/c , po b/c i przed d . Zapisać ich sygnatury.
- 13.3 SSP i ESSP [M].** Wskazać, które pary stanów i pary stan–zdarzenie muszą zostać rozdzielone, aby reachability graph sieci był izomorficzny z TS.
- 13.4 Synteza i brakująca współbieżność [H].** Dodać do procesu oczekiwaną równoległość $b \parallel c$, ale pozostawić w logu tylko jeden przeplot. Pokazać, jak różne abstractions lub assumptions zmieniają wynik; porównać z ograniczeniem opisanym w pracy o Inductive Minerze.

15 Wykład 14: Inductive Miner, process trees i conformance checking

15.1 Cele

Student wykonuje rekurencyjną dekompozycję logu na process tree, rozumie gwarancję soundness modeli blokowych oraz oblicza prosty alignment i interpretuje koszt odchylenia.

15.2 Plan 90 minut

0–15	Process trees: sequence, exclusive choice, parallel, loop, τ .
15–32	Directly-follows graph i pojęcie cut.
32–55	Inductive Miner: znajdź cut, podziel log, rekurencja, fall-through.
55–67	Tłumaczenie process tree na sound workflow net.
67–80	Conformance: token replay versus alignments.
80–90	Koszty moves-on-log/model i cztery wymiary jakości.

15.3 Punkty do omówienia

- Process tree jako hierarchiczny model blokowy.
- Sequence, XOR, AND i loop cuts z relacji directly-follows/start/end.
- Rekurencyjne filtrowanie i projekcja sublogów.
- Dlaczego wynik po translacji jest sound z konstrukcji.
- Niekompletność logu, rzadkie zachowanie i warianty IMf/IMi.
- Alignment jako najtańsza synchronizacja śladu z wykonaniem modelu.
- Synchronous move, move on log, move on model.
- Fitness z kosztu alignmentu; precision jako osobny problem.

15.4 Materiały

- [S. Leemans, D. Fahland, W. van der Aalst, Discovering Block-Structured Process Models](#) — oryginalny Inductive Miner dla zachowania rzadkiego; główne źródło algorytmu.
- [S. Leemans et al., Discovering Process Models from Incomplete Event Logs](#) — czytelny opis process trees, divide-and-conquer i problemu niekompletności.
- [ProM, Background Information on the Inductive Miner](#) — krótkie wyjaśnienie i odсылacz do implementacji.
- [J. Carmona et al., Conformance Checking: Foundations, Milestones and Challenges](#) — rozdział podręcznikowy o conformance i alignments.
- [RWTH Process Mining Lecture 12: Conformance Checking](#) — materiał wideo do przygotowania fragmentu o alignments.

15.5 Ćwiczenia 90 minut

- 14.1 Cuts w directly-follows graph [E].** Dla czterech małych logów rozpoznać kolejno sequence, XOR, parallel i loop cut.
- 14.2 Ręczny Inductive Miner [M].** Dla logu $[\langle a, b, c, e \rangle, \langle a, c, b, e \rangle, \langle a, d, e \rangle]$ skonstruować pełne process tree, pokazując wszystkie sublogi.
- 14.3 Process tree $\rightarrow$ workflow net [M].** Przetłumaczyć wynik na sieć i podać argument soundness indukcyjny względem operatorów drzewa.
- 14.4 Alignment [M].** Dla śladu $\langle a, c, d, e \rangle$ i modelu z zad. 14.2 znaleźć alignment minimalnego kosztu przy koszcie 1 dla ruchu niesynchronicznego.
- 14.5 Porównanie trzech minerów [H].** Dla tego samego logu przewidzieć jakościowo modele Declare, region-based i Inductive Miner: co każdy z nich gwarantuje, co generalizuje i na jaki rodzaj niekompletności jest wrażliwy.

16 Wykład 15: Abstrakcja, słabe bisymulacje i granice rozstrzygalności

16.1 Cele

Student rozumie rolę niewidocznych kroków, rozróżnia strong, weak, branching i stuttering equivalence oraz potrafi czytać mapę wyników rozstrzygalności dla sieci Petriego bez utożsamiania jej z wynikami dla skończonych LTS.

16.2 Plan 90 minut

0–15	Po co ukrywać szczegóły i dlaczego strong bisimulation bywa zbyt drobna.
15–35	Weak transitions $\Rightarrow$, weak bisimulation i obserwowalne ślady.
35–50	Branching bisimulation i zachowanie punktu rozgałęzienia.
50–62	Stuttering equivalence dla struktur Kripkego i $\text{CTL} \setminus X$.
62–75	CEGAR jako operacyjny schemat abstrakcji; powrót do wykładu 9.
75–90	Krajobraz: skończone LTS versus nieskończone grafy osiągalności sieci; nierozstrzygalności.

16.3 Punkty do omówienia

- Ukrycie akcji wewnętrznych i relacja $\xRightarrow{a} = \xrightarrow{\tau^*} \xrightarrow{a} \xrightarrow{\tau^*}$.
- Weak bisimulation może pominać rozgałęzienia wewnętrzne, które branching bisimulation zachowuje.
- Divergence sensitivity jako osobny wybór semantyczny.
- Stuttering equivalence i formuły bez next.
- Relacja abstrakcyjna musi być dobrana do klasy sprawdzanych własności.
- Dla skończonego LTS bisimilarity jest efektywnie rozstrzygalna; trudność dotyczy nieskończonych LTS generowanych przez sieci.
- Strong bisimilarity, weak bisimilarity, trace/language equivalence i simulation mają różne statusy; podawać tabelę z precyzyjną klasą sieci i rodzajem etykietowania.
- Jedna redukcja jako szkic: kodowanie maszyny licznikowej/zero testu przez grę bisymulacyjną; bez technicznych gadżetów pełnego dowodu.

16.4 Materiały

- [J. Esparza, M. Nielsen, Decidability Issues for Petri Nets — a Survey](#) — zaktualizowana mapa własności, równoważności i logik dla sieci Petriego.
- [P. Jančar, High Undecidability of Weak Bisimilarity for Petri Nets](#) — główne źródło do pokazania siły nierozstrzygalności weak bisimilarity.
- [R. van Glabbeek, The Linear Time–Branching Time Spectrum](#) — mapa równoważności i preorderów; źródło uzupełniające.
- [R. De Nicola, F. Vaandrager, Three Logics for Branching Bisimulation](#) — połączenie branching bisimulation z logiką.
- [H. Wimmel, K. Wolf, Applying CEGAR to the Petri Net State Equation](#) — powrót do praktycznej abstrakcji i refinementu.

16.5 Ćwiczenia 90 minut

- 15.1 Strong versus weak [E].** Dla czterech par procesów zdecydować strong i weak bisimilarity; dla odpowiedzi negatywnej podać ruch Spoilera.
- 15.2 Weak versus branching [M].** Skonstruować najmniejszą parę LTS weak-bisimilar, ale nie branching-bisimilar, i wyjaśnić, które rozgałęzienie zostaje utracone.
- 15.3 Stuttering [M].** Zminimalizować strukturę Kripkego względem stuttering equivalence i sprawdzić trzy formuły CTL bez X oraz jedną z X .

- 15.4 Czytanie tabeli rozstrzygalności [M].** Na podstawie survey przypisać 12 problemom status: rozstrzygalny, nierozstrzygalny, zależny od klasy sieci, lub wymagający doprecyzowania. Celem jest wychwycenie niepełnych sformułowań.
- 15.5 Szkic redukcji [H].** Opisać moduły, które redukcja z maszyny dwulicznikowej musi wymusić w grze bisymulacyjnej: increment, decrement, test zera i kara za oszustwo. Nie wymagać pełnego dowodu.

17 Projekt przewodni i organizacja ćwiczeń

17.1 Projekt: proces obsługi zamówienia

Jeden model można rozwijać w kolejnych tygodniach:

1. LTS z akcjami `receive`, `credit`, `stock`, `pay`, `ship`, `cancel`.
2. Specyfikacje LTL/CTL: nie wysyłać bez płatności; każde przyjęte zamówienie zostanie wysłane albo anulowane.
3. Abstrakcja przez symulację/bisymulację: ukrycie kontroli magazynu.
4. Identyfikacja niezależności `credit` i `stock` i grafów zależności.
5. Model w sieci Petriego, inwarianty i unfolding.
6. Model BPMN i analiza soundness.
7. Wygenerowany mały log z rzadką ścieżką anulowania.
8. Trzy modele odkryte: Declare, region-based, Inductive Miner.
9. Alignment dla błędnego śladu `receive`, `ship`, `pay`.

17.2 Zalecany rytm ćwiczeń

- 10 min: krótki quiz diagnostyczny z poprzedniego wykładu;
- 25 min: jedno zadanie techniczne prowadzone wspólnie;
- 35 min: praca w parach nad zadaniem średnim;
- 15 min: dyskusja dwóch rozwiązań i typowych błędów;
- 5 min: sformułowanie zadania domowego/projektowego.

17.3 Narzędzia

- [WoPeD](#) — workflow nets, soundness i wizualizacja token game;
- [ProM](#) — discovery, Declare, region miner, Inductive Miner i conformance;
- [PM4Py](#) — reprodukowalne notebooki do process mining;
- [APT](#) — analiza sieci z wiersza poleceń;
- [LoLA](#) — własności sieci Petriego.

18 Macierz źródeł i przydatność materiałów

Źródło	Format	Ocena	Najlepsze zastosowanie
Winskel, Topics in Concurrency	notes i ćwiczenia	A	LTS, SOS, bisymulacja, HML, podstawy sieci; szkielet pierwszej części kursu.
Cambridge Model Checking Exercises	arkusz zadań	A	modelowanie, symulacja, LTL/CTL, formuły rozdzielające.
Diekert–Muscholl, Trace Theory	rozdział podręcznikowy	A	definicje śladów, grafów zależności i wyników algebraicznych.
Mukund, Trace Logics	slajdy	A-	najlepsze ilustracje logik lokalnych; wymaga selekcji i komentarza prowadzącego.
TUM Petri Nets	pełny kurs	A+	podstawowe źródło dla wykładów 7–10 i ćwiczeń z rozwiązaniami.
Esparza, Unfolding Based MC	slajdy/tutorial	A+	kompletny wykład 8; wysoka jakość przykładów graficznych.
Wimmel–Wolf CEGAR	artykuł	A	bezpośredni most między algebrą sieci i abstrakcją.
OMG BPMN 2.0.2	standard	A/normatywne	definicje notacji; nie używać jako głównego podręcznika dydaktycznego.
Van der Aalst, Workflow	tutorial	A	workflow nets, soundness i zastosowania.
Process Mining Handbook	otwarty podręcznik	A+	współczesne źródło referencyjne dla wykładów 12–14.
Marrella Declare Lab	slajdy	A-	gotowe przykłady templates i LTLf; sprawdzić notację operatorów.
Two-Step Regions	artykuł	A	dokładnie odpowiada koncepcji automat/TS $\rightarrow$ regiony $\rightarrow$ sieć.
Leemans et al., Inductive Miner	artykuł	A	process trees, cuts i gwarancja soundness.
Esparza–Nielsen survey	survey	A	finalowa mapa rozstrzygalności; wymaga precyzyjnego określenia wariantu problemu.

Ocena A oznacza źródło, które może stanowić bezpośrednią podstawę slajdów i przygotowania zadań; A- wymaga większej redakcji lub ujednolicenia notacji; A+ jest materiałem szczególnie kompletnym i dydaktycznie spójnym.

19 Niezależna recenzja ekspercka programu

19.1 Metoda oceny

Program oceniono ponownie po jego ułożeniu, przyjmując sześć kryteriów: spójność narracji, obciążenie pojęciowe, adekwatność ćwiczeń, reprezentatywność teorii współbieżności, siła części aplikacyjnej oraz aktualność i dostępność źródeł.

19.2 Mocne strony

- [OK] Wyraźna historia intelektualna.** Kurs nie jest katalogiem modeli: każdy kolejny formalizm odpowiada na konkretny problem poprzedniego.
- [OK] Prawdziwe połączenie teorii i praktyki.** Workflow/process mining nie jest dodatkiem; wykorzystuje LTL, automaty, regiony, sieci i własności behawioralne.
- [OK] Dobre jądro algorytmiczne.** Coverability, WQO, inwarianty i unfolding są wykładalne, mają czytelne przykłady i nie wymagają ukrywania zasadniczych argumentów.
- [OK] Ćwiczenia są równoważne rangą wykładom.** Obejmują dowody, ręczne algorytmy, modelowanie i pracę narzędziową.

[OK] Źródła są mocne. Rdzeń pochodzi z pełnych kursów Cambridge i TUM oraz uznanych tutoriali i podręcznika process mining.

19.3 Ryzyka i konieczne korekty

- [!]** Wykład 2 jest zbyt szeroki, jeżeli studenci nie znają logik temporalnych. Nie należy dowodzić obu pełnych algorytmów LTL i CTL. Rekomendacja: CTL bottom-up szczegółowo, LTL automata-theoretic tylko jako schemat albo odwrotnie.
- [!]** Wykład 6 może zerwać narrację. Zbyt wiele logik śladów obniży zrozumienie. Rekomendacja: 45 minut na semantykę liniaryzacji, 25 minut na jedną logikę lokalną, 10 minut na FO/pełność, 10 minut na przykłady.
- [!]** Wykład 9 nie może stać się kursem ILP. Równanie stanu i refinement mają być interpretowane strukturalnie; solver jest narzędziem, nie tematem.
- [!]** Wykład 14 łączy dwa duże tematy. Inductive Miner powinien zająć około 60 minut, a alignment 30 minut. Pełne miary precision należy zostawić do ćwiczeń lub projektu.
- [!]** Wyniki nierozstrzygalności są podatne na błędne skróty. Zawsze trzeba podać: strong/weak, simulation/bisimulation, labelled/unlabelled, dwie sieci czy sieć kontra skończony system, oraz klasę sieci.

19.4 Ocena obciążenia

Blok	Ocena	Komentarz
LTS, logiki, bisymulacja (1–4)	odpowiedni	Cztery spotkania dają wystarczające podstawy i język abstrakcji.
Ślady i logiki śladów (5–6)	wysoki	Dobry dla kursu magisterskiego; na poziomie licencjackim wykład 6 powinien być przeglądowy.
Sieci i weryfikacja (7–10)	odpowiedni/amb	Najbardziej spójny blok; cztery wykłady są minimalnym sensownym wymiarem.
Workflow i mining (11–14)	odpowiedni	Cztery spotkania pozwalają pokazać zastosowanie bez sprowadzenia go do demonstracji narzędzia.
Abstrakcja i granice (15)	wysoki	Dobry finał, o ile CEGAR został już przeciwiczony w wykładzie 9.

19.5 Rekomendowany rdzeń egzaminacyjny

- definicje LTS/Kripke, LTL/CTL, symulacji i bisymulacji;
- trace equivalence, graf zależności i Foata normal form;
- semantyka sieci, boundedness, deadlock, coverability;
- P-invariant, równanie stanu, trap/siphon;
- konfiguracja i complete finite prefix na małym przykładzie;
- workflow net i soundness;
- zasada działania Declare, region miner i Inductive Miner;
- alignment na małym przykładzie;

- różnica strong/weak/branching bisimulation i świadomość granic rozstrzygalności.

Nie należy wymagać na egzaminie dowodu pełności logiki śladów, algorytmu ogólnego reachability ani technicznego dowodu nierozstrzygalności bisymulacji sieci.

19.6 Werdykt

Ocena końcowa: 8.5/10 dla kursu magisterskiego i 7/10 dla kursu licencjackiego bez redukcji zakresu.

Po ograniczeniu wykładu 6 do jednego formalizmu, pozostawieniu reachability jako panoramy oraz przeniesieniu części conformance do ćwiczeń plan jest realistyczny dla 15×90 minut. Największą wartością kursu jest wspólna oś: pojęcie obserwowalnego zachowania i różne sposoby usuwania nieistotnej kolejności lub stanu.

20 Uwagi o prawach do materiałów i sposobie użycia

- Linki służą jako materiały do przygotowania własnego wykładu. Ćwiczenia w tym dokumencie są streszczeniami lub adaptacjami, nie kopiami arkuszy.
- Materiały W. van der Aalsta są udostępniane do użycia dydaktycznego z wymaganiem wyraźnego wskazania źródła; użycie oryginalnych plików PowerPoint wymaga osobnej prośby opisanej na stronie kursów.
- Przy przenoszeniu rysunków lub slajdów do własnych materiałów należy sprawdzić licencję konkretnego pliku i umieścić autora oraz link na slajdzie.
- Najbezpieczniejszy wariant to przerysowanie przykładów w jednolitej notacji i podanie „na podstawie” wraz z odsyłaczem.

21 Szybka lista najważniejszych linków

1. [Winskel: Topics in Concurrency](#)
2. [Cambridge: Exercises for Model Checking](#)
3. [Diekert–Muscholl: Trace Theory](#)
4. [Mukund: A Quick Tour of Trace Logics](#)
5. [TUM: pełny kurs Petri Nets](#)
6. [Esparza: Unfolding Based Model Checking](#)
7. [Wimmel–Wolf: CEGAR and the State Equation](#)
8. [OMG BPMN 2.0.2](#)
9. [Van der Aalst: Process Mining Courses](#)
10. [Process Mining Handbook](#)
11. [Two-Step Region-Based Process Mining](#)
12. [Inductive Miner](#)
13. [Esparza–Nielsen: Decidability Issues for Petri Nets](#)

Koniec dokumentu. Linki sprawdzono w trakcie opracowania 31 lipca 2026 r.